PROJECT DOCUMENTATION

Sketch2Shape AI

Turning a single hand-drawn sketch into a textured, downloadable 3D model (.glb) using open-source generative AI.

Sketch → Understand → Reference image → Isolate → 3D mesh → Validate → .glb

TEAM

Manashviruti

TechZephyr 4.0
IIT Bhubaneswar
November 2026

Contents

1 Problem statement and objective

2 Solution overview and features

3 System architecture

4 The generation pipeline, step by step

5 Model choices and rationale

6 Technology stack and dependencies

7 Backend API

8 Frontend and user experience

9 Installation

10 Running the application

11 Inputs and outputs

12 Hardware and computational resources

13 Generation time and sample results

14 Robustness and error handling

15 Limitations and issues encountered

16 Future work

17 Credits and licences

1 Problem statement and objective

Designers, students, game developers and makers usually begin with a quick sketch. Turning that sketch into a 3D model by hand takes hours of skilled work in tools such as Blender.

Sketch2Shape AI automates this step. From **one single-view sketch** (drawn in the browser or uploaded as an image) and an optional text description, it produces a **textured 3D model in the standard `.glb` format**. The user can inspect the model in an interactive viewer and download it for use in other software.

Objectives

- **Sketch to 3D asset:** accept a 2D sketch plus an optional prompt and return a textured, valid GLB file.
- **Generalise to unseen sketches:** nothing is tuned to particular objects. A vision model interprets every new sketch and writes a prompt for it.
- **Interactive preview:** show the result in a 3D viewer in the browser and allow download.
- **Transparency:** show live progress and every intermediate result, so the user understands what the AI is doing.
- **Free and open:** use only open-source models and free service tiers, with no paid APIs.
- **Robustness:** validate every stage, explain failures in plain language, and provide offline fallbacks.

Scope

The system targets **single, standalone objects** (furniture, household items, vehicles, gadgets, animals, toys) drawn from one viewpoint. Whole scenes, rooms and photographs of people are out of scope and are rejected with a clear message.

2 Solution overview and features

A web application with a **Next.js** frontend and a **FastAPI** backend. The backend runs a six-step AI pipeline for every request. The core idea is to **bridge the gap between line art and 3D**. Image-to-3D models are trained on real objects, not drawings, so the sketch is first turned into a realistic reference image that follows its shape. That image is then reconstructed in 3D.

Draw or upload

A full-screen, Excalidraw-style whiteboard (pen, eraser, pan, undo/redo, keyboard shortcuts), or upload, drag-and-drop or paste a PNG/JPG sketch.

Describe (optional)

Free-text description of the object, materials, colours and style. The user's text overrides the AI's guess.

Live progress

Six-step progress tracker with live messages (denoising steps, GPU queue position) and thumbnails of each intermediate image.

Interactive 3D viewer

Orbit, zoom, pan, auto-rotate, wireframe and reset view, with studio lighting and soft shadows.

One-click download

Saves a named `.glb` file, plus a mesh report and model facts (triangles, material, size, engine used).

Safe and robust

Input validation, non-sketch rejection, quality checks after every step, and automatic fallbacks when an online service is unavailable.

Key design decision. Feeding a raw sketch directly to an image-to-3D model gives flat, paper-like results. Generating an intermediate photoreal image guided by the sketch through ControlNet keeps the user's shape and gives the 3D model the kind of input it was trained on.

3 System architecture

Components

Frontend: Next.js 16 · React 19

- Whiteboard and upload composer
- Sends the sketch with `POST /api/generate`
- Polls `GET /api/status/{id}` every 3 s
- Progress card with intermediate images
- 3D viewer (three.js / React Three Fiber)
- Download .glb

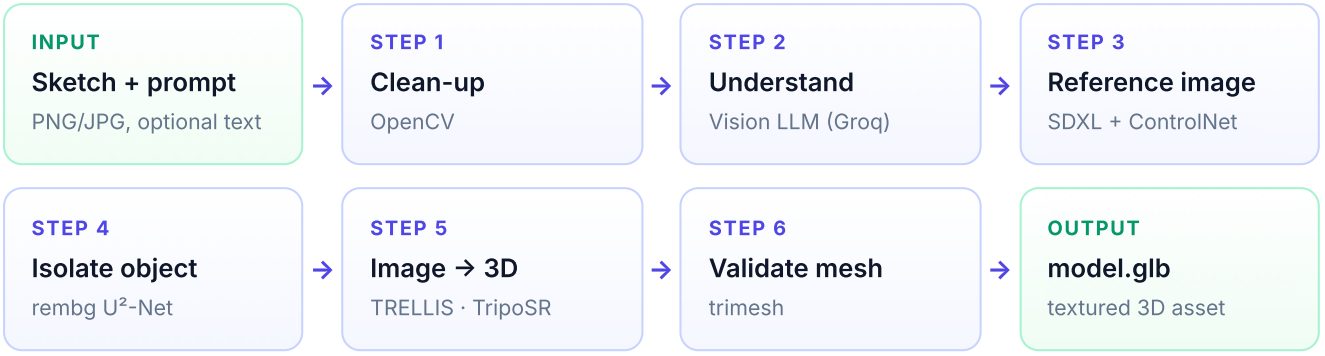
→ sketch + prompt

← job_id, status, files

Backend: FastAPI · Python 3.12

- Validates the upload and creates a job folder
- Runs the pipeline as a background task
- Thread-safe in-memory job store (+ `job.json`)
- Serves job files under `/files`
- Local models: OpenCV, SDXL, rembg, TripoSR, trimesh
- Remote: Groq (LLM), Hugging Face Space (TRELLIS)

Pipeline



Asynchronous job design

Generating a model takes minutes, far longer than a sensible HTTP request. The backend therefore uses a **job pattern**:

- `POST /api/generate` validates the upload, stores it in a new job folder and immediately returns `202 Accepted` with a `job_id`.
- The pipeline runs as a FastAPI background task. Heavy work (image processing, diffusion, 3D) runs in worker threads, so the server keeps answering status requests.
- Each step updates the job's *stage*, *progress* (0–100) and a live *message*, and registers its output files as *artifacts*.
- The frontend polls `GET /api/status/{job_id}` every 3 seconds until the job is `completed` or `failed`.
- Every file a job produces is served at `/files/jobs/<job_id>/<file>` (with CORS for the frontend).
- Only one diffusion run uses the GPU at a time, which avoids out-of-memory errors.

4 The generation pipeline, step by step

Step 1: Sketch clean-up OpenCV · local CPU

The upload is converted to grey-scale and binarised with an adaptive threshold, which copes with uneven lighting on photographed paper. Dark-background drawings are inverted automatically and small specks are removed. The drawing is then cropped, centred and padded onto a 1024 × 1024 white square. The **ink ratio** (share of drawn pixels) is measured and reused later. Output: `sketch_clean.png`.

Step 2: Sketch understanding Vision LLM · Groq API

A vision-language model receives the original upload, the cleaned sketch and the user's description. It must answer with **strict JSON** that matches a schema: whether this is a sketch of one object, the object category and description, the level of detail, key features, materials, colours, a ready-to-use image prompt and extra negative prompts. The reply is validated with Pydantic.

- Instructions are turned into attributes ("make it red" becomes "a red plastic pencil sharpener"), and colours apply to the object only.
- Watermarks and stray text are ignored, and solid, realistic materials are preferred.
- Photos of people, scenes and blank images are **rejected** with an explanation.
- The level of detail sets the ControlNet strength: 0.55 sparse, 0.70 moderate, 0.85 detailed.
- If the LLM is unavailable (no key, rate limit, timeout), a **rule-based fallback** builds a prompt from the user's text and the measured ink ratio.

Output: `enrichment.json` (analysis, final prompt and negative prompt).

Step 3: Reference image generation SDXL + ControlNet · local GPU

Stable Diffusion XL with a **Scribble ControlNet** turns the sketch into a photoreal product shot. ControlNet keeps the drawn outline; the prompt decides materials and colours. A fixed style suffix asks for a single solid object, centred, on a seamless pure-white background with soft studio light. A negative prompt blocks coloured backgrounds, text, glow and flat 2D looks. The sketch is inverted to white lines on black, which is what the ControlNet expects. Output is checked for blank or invalid images. Output: `control.png`, `generated.png` (1024^2).

Step 4: Object isolation rembg U²-Net · local CPU

rembg predicts an alpha mask for the main object. The mask is then cleaned: separate small regions (steam, sparkles, dust) are dropped, tiny pin-holes inside the object are filled, and genuine openings such as a mug handle are kept. The object is cropped, centred and padded to a square with a margin, which is the framing image-to-3D models expect. The step fails clearly if nothing, or the whole image, is detected. Output: `foreground.png` (transparent), `foreground_white.png`.

Step 5: 3D reconstruction TRELLIS / TripoSR

The isolated object image is reconstructed into a textured mesh. Providers are tried in order:

- **TRELLIS** (primary) on the public Hugging Face Space `trellis-community/TRELLIS`, called with `gradio_client` on the free ZeroGPU tier. It produces a mesh with a PBR material and a 1024² texture. The UI shows the queue position while waiting.
- **TripoSR** (fallback) runs locally on the laptop: one forward pass, then marching cubes. It produces a mesh with vertex colours and needs no internet or quota.

If TRELLIS fails for any reason (quota used up, queue full, Space down, timeout), the job switches to TripoSR automatically. After a quota error the Space is skipped for 30 minutes. Every attempt and its error are recorded in the job details. Output: `model_raw.glb`, `trellis_preview.mp4`.

Step 6: Mesh validation and clean-up `trimesh · local CPU`

- **Hard checks** (the job fails if any is broken): the file loads as glTF; at least one mesh with ≥ 10 vertices and ≥ 20 faces; all coordinates finite; the object has real size and is not flat.
- **Soft checks** (warnings only): watertightness (*not required*, since single-view models are often open underneath), consistent winding, presence of texture or colours, very large triangle counts.
- **Clean-up:** removes zero-area and duplicate triangles and unused vertices without touching UVs, textures or materials.
- **Normalisation:** centred on X/Z, resting on Y = 0, longest side scaled to 1 m.
- **Safety net:** the cleaned file is re-loaded and compared with the original (meshes, faces, textures, vertex colours). If anything was lost, the original file is shipped unchanged, so materials are always preserved.

Output: `model.glb` (final asset), `validation.json` (mesh report).

5 Model choices and rationale

STEP	MODEL	WHY IT WAS CHOSEN	LICENCE
Understand	Qwen vision-language model (`qwen/qwen3.8-27b`) via Groq	Accepts images and supports strict JSON-schema output. Groq's free tier answers in about a second. Any OpenAI-compatible endpoint can be used instead (`LLM_PROVIDER=openai`).	Open weights
Reference image	SDXL base 1.0 + ControlNet Scribble SDXL (xinsir) + SDXL VAE fp16-fix	Scribble ControlNet is designed for rough hand-drawn lines, so it follows the outline without copying the pencil look. SDXL gives 1024^2 photo quality. The fp16-fix VAE prevents black or NaN images in half precision on Apple GPUs.	Open RAIL++-M · Apache-2.0 · MIT
Isolate	rembg with U^2-Net	Reliable salient-object masks in 1–2 s on CPU, a small model (~170 MB), and easy to swap for sharper models.	MIT · Apache-2.0
Image → 3D	TRELLIS (Microsoft)	One of the strongest open image-to-3D models, with textured PBR output. It needs a large NVIDIA GPU, which the free Hugging Face ZeroGPU tier provides.	MIT
Image → 3D (fallback)	TripoSR (Stability AI + Tripo AI)	Single forward pass, small enough for a laptop, works offline. Its CUDA-only marching-cubes dependency is replaced by a scikit-image version, so nothing has to be compiled on macOS.	MIT
Validation	trimesh	Standard Python mesh library that reads and writes glTF while keeping PBR materials.	MIT
Viewer	three.js · React Three Fiber · drei	Native glTF/GLB support. The studio lighting is built in code, so the viewer works offline.	MIT

Changes from the suggested tools

- **Groq + Qwen instead of GPT-4o-mini:** no paid API credits were available, and the project aims to be free and open. OpenAI remains supported by configuration.
- **SDXL running locally instead of a hosted image API:** hosted SDXL ControlNet Scribble endpoints were paid or unavailable. Running locally is free and private, at the cost of time and ~10 GB of model downloads.
- **Cloud GPU for 3D with a local fallback:** TRELLIS needs an NVIDIA data-centre-class GPU, so it uses the free Hugging Face ZeroGPU tier. TripoSR guarantees a result when the daily quota is used up.

Alternatives considered

- **Hunyuan3D 2.1:** strong geometry, but its licence is not fully open (territory and user-count restrictions). It needs ~29 GB of NVIDIA VRAM and several times more GPU time per model, so the free quota would cover far fewer models.
- **TRELLIS.2** (MIT): a newer, higher-quality successor, planned as the next upgrade (Section 16).
- **Sketch directly to 3D** (no reference image): rejected after testing, because it gives flat, paper-like geometry.

6 Technology stack and dependencies

LAYER	TECHNOLOGIES
Frontend	Next.js 16 (App Router), React 19, TypeScript 5, Tailwind CSS v4, three.js 0.175, @react-three/fiber 9, @react-three/drei 10
Backend	Python 3.12, FastAPI, Uvicorn, Pydantic v2, python-multipart, python-dotenv
Image processing	OpenCV (headless), NumPy, Pillow, scikit-image
AI / ML	PyTorch (MPS / CUDA / CPU), diffusers, transformers, accelerate, safetensors, huggingface_hub, rembg + onnxruntime
3D	gradio_client (TRELLIS Space), TripoSR (`tsr` package) + omegaconf + einops, trimesh, imageio
LLM client	openai Python SDK (OpenAI-compatible, used for Groq)

Exact Python dependencies are listed in `backend/requirements.txt`, and JavaScript dependencies in `frontend/package.json`.

Project structure

```
sketch2shape-ai/
├── README.md
├── docs/                          API reference, this document, diagrams
├── backend/
│   ├── app/
│   │   ├── main.py, routes.py     application setup and API endpoints
│   │   ├── pipeline.py            runs steps 1–6 for each job
│   │   ├── jobs.py, schemas.py    job store and data models
│   │   ├── config.py, errors.py   settings (.env) and user-facing errors
│   │   └── services/
│   │       ├── image_preprocessing.py     step 1
│   │       ├── semantic_enrichment.py     step 2
│   │       ├── image_generation.py        step 3
│   │       ├── background_removal.py      step 4
│   │       ├── reconstruction*.py         step 5 (TRELLIS + TripoSR)
│   │       └── mesh_validation.py         step 6
│   ├── scripts/                   download_models.py, setup_triposr.py
│   ├── requirements.txt
│   └── .env.example
└── frontend/
    └── src/
        ├── app/page.tsx           main page
        ├── components/            Whiteboard, ProgressCard, ResultCard, ModelViewer
        ├── hooks/useJobStatus.ts polling hook
        └── lib/                   API client and helpers
```

7 Backend API

METHOD	PATH	PURPOSE
GET	`/api/health`	Server status, image-model loading state, 3D provider readiness
POST	`/api/generate`	Upload `sketch` (PNG/JPG ≤ 10 MB) and optional `prompt` (≤ 500 chars). Returns `202` with `job_id`
GET	`/api/status/{job_id}`	Status, stage, progress, live message, error, artifact URLs, per-step details
GET	`/files/jobs/{job_id}/{file}`	Download any file produced by the job (e.g. `model.glb`)

Job lifecycle

FIELD	VALUES
status	`queued` → `processing` → `completed` or `failed`
stage	`received`, `preprocessing`, `understanding`, `generating_image`, `removing_background`, `reconstructing`, `validating`, `done`
progress	10 → 20 → 30–55 (per denoising step) → 58 → 62–90 → 92 → 100
artifacts	`input`, `sketch_clean`, `enrichment`, `control_image`, `generated_image`, `foreground`, `model_raw`, `preview_video`, `model_glb`, `validation`

Error responses

CODE	WHEN
400	Empty file, unreadable image, or prompt too long
404	Unknown job ID (also after a server restart, since jobs are kept in memory)
413	File larger than the upload limit
415	File is not PNG or JPEG
422	Missing `sketch` field

A job that fails during the pipeline still returns `200` from the status endpoint, with `status: "failed"` and a plain-language `error`. A full reference with example responses is in `docs/API.md`. Interactive documentation is available at `http://localhost:8000/docs` while the server runs.

8 Frontend and user experience

- **Home page:** a chat-style composer with suggestion chips, a sketch thumbnail, an auto-growing description box and a *Generate 3D Preview* button (also ⌘/Ctrl + Enter).
- **Whiteboard:** a full-screen dotted canvas with pen, whole-stroke eraser, pan (Space / H / middle mouse), three pen sizes, undo/redo and clear. Drawings are exported as a centred, square, black-on-white PNG.
- **Upload:** file picker, drag-and-drop anywhere on the page, or paste from the clipboard. Type and size are checked before sending.
- **Progress card:** progress bar with elapsed time, six steps marked done / active / failed, the backend's live message, and thumbnails of the clean sketch, AI reference and isolated object as they appear.
- **Result card:** 3D viewer (auto-rotate, wireframe, reset view), object name and description, *Download .glb*, model facts (engine, triangles, vertices, material, size, file size, total time), warnings, and links to the mesh report and turntable video.
- **Errors:** plain-language reason with *Try again* and *Edit sketch or prompt* actions. If the backend stops responding, polling stops after a few attempts with a clear message.
- **Resilience:** the job ID is kept in the page URL, so a refresh reopens the same job. Generate is disabled while a job runs.

9 Installation

Prerequisites: **Python 3.10–3.12**, **Node.js 20+**, **git** (used by the TripoSR setup script), about **15 GB** of free disk space, and an internet connection for the first setup.

Backend

```
cd sketch2shape-ai/backend
python3 -m venv .venv
source .venv/bin/activate              # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env                    # then add your keys (below)
python -m scripts.download_models       # one time: ~12 GB of open-source models
python -m scripts.setup_triposr         # one time: offline 3D fallback + self-test
```

API keys (free, optional)

VARIABLE	PURPOSE	WHERE TO GET IT
`GROQ_API_KEY`	Sketch understanding (step 2)	console.groq.com/keys
`HF_TOKEN`	More free GPU time for TRELLIS (step 5), using a "Read" token	huggingface.co/settings/tokens

Without these keys the app still works, using a rule-based prompt and the offline TripoSR model.

Frontend

```
cd ../frontend
npm install
cp .env.local.example .env.local        # NEXT_PUBLIC_API_URL=http://localhost:8000
```

10 Running the application

Start the backend and the frontend in two terminals.

```
# Terminal 1: backend
cd sketch2shape-ai/backend
source .venv/bin/activate
uvicorn app.main:app --port 8000

# Terminal 2: frontend
cd sketch2shape-ai/frontend
npm run dev
```

1. Wait for `Application startup complete`. The image model finishes loading in the background; `http://localhost:8000/api/health` reports `"status": "ready"` when it is ready.
2. Open **http://localhost:3000**.
3. Draw on the whiteboard or upload a sketch, and optionally describe the object.
4. Click **Generate 3D Preview** and follow the live progress.
5. Inspect the model in the viewer and click **Download .glb**.

> **Tip.** Do not start the backend with `--reload` during a demo. A reload restarts the server, forgets running jobs and reloads the image model.

Main settings (`backend/.env`)

SETTING	DEFAULT	EFFECT
`SD_STEPS`	30	Reference-image quality vs. speed
`IMAGE_SIZE`	1024	Lower to 768 if memory is tight
`RECON_PROVIDERS`	trellis,triposr	3D provider order; `triposr` alone works fully offline
`TRELLIS_SS_STEPS` / `TRELLIS_SLAT_STEPS`	20 / 20	TRELLIS sampling steps (12 is fastest)
`TRELLIS_TIMEOUT_SECONDS`	300	Switch to the fallback after this long
`REJECT_NON_SKETCHES`	true	Reject photos of people and scenes
`MESH_NORMALIZE` / `MESH_TARGET_SIZE`	true / 1.0	Centre, ground and scale the final model

Every setting is documented in `backend/.env.example`.

11 Inputs and outputs

Input

FIELD	REQUIREMENTS
Sketch	PNG or JPEG, up to 10 MB. One object, single view. Dark lines on light paper (light on dark is inverted automatically).
Prompt (optional)	Up to 500 characters describing the object, materials, colours and style.

> **For best results:** draw one object at a **three-quarter angle** (showing front, side and top), use closed outlines, avoid text and watermarks, and describe the object (*"a red plastic pencil sharpener"*) rather than giving an instruction (*"make it red"*).

Output

All files are saved in `backend/data/jobs/<job_id>/` and served at `http://localhost:8000/files/jobs/<job_id>/`.

FILE	DESCRIPTION
`model.glb`	**Final 3D asset.** Binary glTF 2.0 triangle mesh with a PBR material and 1024² base-colour texture (TRELLIS) or vertex colours (TripoSR). Y-up, centred, resting on the ground, longest side 1 m. Opens in Blender, Unity, Unreal, three.js and standard glTF viewers.
`model_raw.glb`	The mesh exactly as returned by the 3D model, before validation
`validation.json`	Mesh report: counts before/after, removed items, watertight, winding, size, area, texture info, warnings
`enrichment.json`	The vision model's analysis and the exact image prompt and negative prompt used
`sketch_clean.png`, `control.png`	Cleaned sketch and the ControlNet input
`generated.png`	AI reference image (1024 × 1024)
`foreground.png`, `foreground_white.png`	Isolated object (transparent / on white)
`trellis_preview.mp4`	Turntable video (when TRELLIS was used)
`job.json`	Full job record: status, timings per step, 3D provider used and any fallback reason

12 Hardware and computational resources

RESOURCE	DETAILS
Development machine	Apple Silicon MacBook Pro (arm64), 24 GB or more unified memory, macOS
Local GPU	Apple GPU through PyTorch MPS, half precision (fp16). An NVIDIA GPU with ≥ 12 GB VRAM is used automatically if present (CUDA). CPU-only works but is very slow for step 3.
Local memory use	The SDXL pipeline needs roughly 10–12 GB of memory while generating. TripoSR needs a few GB more when used.
Remote: language model	Groq API, free tier (step 2)
Remote: 3D	Hugging Face ZeroGPU, a shared NVIDIA data-centre GPU, free tier with a personal token (step 5, TRELLIS)
Disk	About 15 GB for model weights (SDXL + ControlNet + VAE ≈ 10–12 GB, TripoSR ≈ 1.7 GB, DINO ≈ 0.35 GB, U²-Net ≈ 0.17 GB), downloaded once
Network	Needed for first setup, Groq and the TRELLIS Space. Without it, the app runs with the rule-based prompt and local TripoSR.
Software	Python 3.12.7, PyTorch 2.15, Node.js 20+, Next.js 16

13 Generation time and sample results

Measured on the development MacBook with `SD_STEPS=25`, the setting used for the demo.

Case 1: TRELLIS on the free Hugging Face GPU (main path)

STEP	RUNS ON	SPORTS CAR	MUG
1 · Clean-up	Laptop CPU	0.02 s	0.05 s
2 · Understand	Groq API	1.7 s	1.2 s
3 · Reference image (SDXL, 25 steps)	Laptop GPU	105 s	132 s
4 · Isolate object	Laptop CPU	2.3 s	1.1 s
5 · Image → 3D (TRELLIS, incl. queue)	HF ZeroGPU	56 s	52 s
6 · Validate mesh	Laptop CPU	0.3 s	0.1 s
Total, sketch → .glb		**2 min 45 s**	**3 min 07 s**

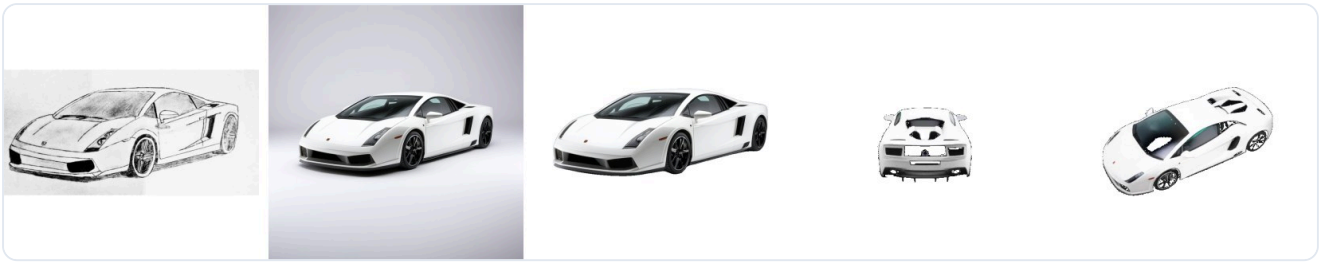
Case 2: TripoSR fallback on the laptop

STEP	SPORTS CAR	MUG
3 · Reference image (SDXL, 25 steps)	105 s	103 s
5 · Image → 3D (incl. a failed TRELLIS attempt of ~16 s due to quota)	24 s	30 s
Total, sketch → .glb	**2 min 13 s**	**2 min 17 s**

Step 3 on the laptop GPU accounts for 60–75% of the total. TRELLIS gives textured, cleaner models (10–18k triangles). TripoSR is faster but gives vertex colours only and denser meshes (86–118k triangles). On an NVIDIA RTX 4090-class GPU, step 3 would typically take about 10 s.

Sample results

Sketch → AI reference image → isolated object → generated 3D model (two views).



Sports car: 10,492 triangles, PBR texture 1024², 1.6 MB, 2 min 45 s.



Mug (drawn on the app's whiteboard): 18,114 triangles, PBR texture 1024², 1.5 MB, 3 min 07 s. Both models are included in the `samples/` folder.

14 Robustness and error handling

SITUATION	BEHAVIOUR
Invalid upload	Wrong type, empty, unreadable or too large files are rejected immediately with a specific HTTP error and message.
Not a sketch	Photos of people, scenes and blank images are rejected in step 2 with the model's reason.
LLM unavailable	Up to 4 automatic retries that honour rate-limit headers, then a rule-based prompt. The job continues with a warning.

Blank or broken image	Step 3 detects blank outputs and asks the user to retry (a new random seed is used).
Bad cut-out	Step 4 fails clearly if no object, or the whole image, is detected.
3D service unavailable	Automatic switch from TRELLIS to local TripoSR. A quota error puts the Space on a 30-minute cool-down. All attempts are recorded.
Invalid mesh	Empty, non-finite or flat meshes fail with an explanation. Clean-up never loses materials (original shipped if needed).
Backend unreachable	The frontend retries polling, then shows a clear "lost contact" message instead of hanging.
Viewer problem	An error boundary shows a message in the viewer area, and the download still works.

15 Limitations and issues encountered

Issues encountered while generating models

ISSUE	RESOLUTION
The free Hugging Face ZeroGPU quota ran out after bursts of TRELLIS runs ("exceeded your free ZeroGPU quota... try again in 23:23:09"), and some jobs failed.	Automatic fallback to local TripoSR, plus a 30-minute cool-down for the Space after a quota error.
TripoSR would not load with the latest `transformers` library (renamed ViT weight names), so the fallback also failed.	Fixed: weight names are converted at load time. The TripoSR self-test now passes.
Groq's free-tier rate limit occasionally forced the basic prompt, which gave weaker results.	Automatic retries, and the basic prompt now rewrites instructions into object descriptions.
Colour instructions ("make it red") tinted the whole image, including the background.	Prompts keep colours on the object and enforce a seamless white background. Coloured backgrounds, glow and sparkles are in the negative prompt.
The 3D viewer's loading indicator caused React warnings when unmounting.	Replaced with a standard page overlay outside the 3D canvas.

Known limitations

- **Hidden sides are inferred:** only one view is given, so the back and underside are generated. Straight-on views (for example a laptop seen from the front) give flatter depth.
- **Reinterpretation:** the reference image may slightly change proportions or simplify small details.
- **Thin and transparent parts** (wires, glass, watch hands) may be thickened, merged or missing.
- **Not watertight:** fine for viewing, games and AR. 3D printing needs a repair step.
- **Normalised scale:** the longest side is always 1 m, not the real-world size.
- **Speed:** about 2–3 minutes per model on a laptop, dominated by local image generation. Jobs run one at a time.
- **Free services:** Groq and Hugging Face may rate-limit or queue. The fallbacks keep the app working at lower quality (TripoSR gives simpler shapes and vertex colours).
- **Prototype backend:** jobs are stored in memory and lost on restart (files remain on disk). There is no authentication. Tested on macOS with Apple Silicon.

16 Future work

TRELLIS.2 as the main 3D model

MIT-licensed successor with better geometry and up to 4096² PBR textures.

Best-of-N reference images

Generate several reference images and let the user (or a scorer) pick one before 3D.

Persistent jobs and queue

Store jobs in a database and queue requests for multiple users.

Print-ready export

Optional watertight repair (hole filling, solidify) as a second download.

Fully offline mode

A local vision model (e.g. via Ollama) for step 2.

Multi-view sketches

Accept front and side sketches together for more accurate hidden geometry.

17 Credits and licences

Third-party models and libraries keep their own licences:

COMPONENT	LICENCE
Stable Diffusion XL base 1.0 (Stability AI)	CreativeML Open RAIL++-M
ControlNet Scribble SDXL (xinsir)	Apache-2.0
SDXL VAE fp16-fix (madebyollin)	MIT
rembg · U²-Net	MIT · Apache-2.0
TRELLIS (Microsoft), Space by trellis-community	MIT
TripoSR (Stability AI and Tripo AI) · DINO ViT-B/16 (Meta)	MIT · Apache-2.0
Qwen (Alibaba), served by Groq	Qwen licence (open weights)
trimesh, three.js, React Three Fiber, drei, Next.js, FastAPI	MIT

Sketch2Shape AI · Team Manashviruti · TechZephyr 4.0, IIT Bhubaneswar